```
48
49  int main()
50  {
51      cin >> p1 >> p2 >> p3 >> s;
52      n = s.size();
53
54      for(int i = 0; i < n; i++)
55      {
56          char ch = s[i];
57
58          if(s[i] != '-' || i == 0 || i == n - 1) ret += ch;
59          else
60          {
61              char left = s[i - 1], right = s[i + 1];
62              // 判断是否展开
63              if(isdig(left) && isdig(right) && right > left ||
64                 islet(left) && islet(right) && right > left)
65              {
66                  // 展开
67                  add(left, right);
68              }
69              else
70              {
71                  ret += ch;
72              }
73          }
74      }
75
76      cout << ret << endl;
77
78      return 0;
79  }
```

2. 高精度

当数据的值特别大，各种类型都存不下的时候，此时就要用高精度算法来计算加减乘除：

- 先用字符串读入这个数，然后用数组逆序存储该数的每一位；
- 利用数组，模拟加减乘除运算的过程。

高精度算法本质上还是模拟算法，用代码模拟小学列竖式计算加减乘除的过程。

2.1 高精度加法

> **题目来源：** 洛谷
>
> **题目链接：** P1601 A+B Problem（高精）
>
> **难度系数：** ★

【题目描述】

> 高精度加法，相当于 $a+b$ problem，**不用考虑负数**。

【输入描述】

> 分两行输入。
>
> $a,b \leq 10^{500}$ 。

【输出描述】

> 输出只有一行，代表 $a+b$ 的值。

【示例一】

> 输入：
>
> 1
>
> 1
>
> 输出：
>
> 2

【示例二】

> 输入：
>
> 输出：

【示例三】

> 输入：
>
> 1001
>
> 9099
>
> 输出：
>
> 10100

【解法】

模拟小学「列竖式」计算「两数相加」的过程。



1. 用字符串读入数据；
2. 将字符串的每一位拆分，逆序放在数组中；
3. 模拟列竖式计算的过程：
   a. 对应位累加；
   b. 处理进位；
   c. 处理余数。
4. 处理结果的位数。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int a[N], b[N], c[N];
int la, lb, lc;

// 高精度加法的模版 - c = a + b;
void add(int c[], int a[], int b[])
{
    for(int i = 0; i < lc; i++)
    {
        c[i] += a[i] + b[i]; // 对应位相加，再加上进位
        c[i + 1] += c[i] / 10; // 处理进位
        c[i] %= 10; // 处理余数
    }

    if(c[lc]) lc++;
}
```

```cpp

int main()
{
    string x, y; cin >> x >> y;

    // 1. 拆分每一位，逆序放在数组中
    la = x.size(); lb = y.size(); lc = max(la, lb);
    for(int i = 0; i < la; i++) a[la - 1 - i] = x[i] - '0';
    for(int i = 0; i < lb; i++) b[lb - 1 - i] = y[i] - '0';

    // 2. 模拟加法的过程
    add(c, a, b); // c = a + b

    // 输出结果
    for(int i = lc - 1; i >= 0; i--) cout << c[i];

    return 0;
}
```

2.2 高精度减法

> **题目来源：** 洛谷
>
> **题目链接：** P2142 高精度减法
>
> **难度系数：** ★

【题目描述】

> 高精度减法。

【输入描述】

> 两个整数 a, b（第二个可能比第一个大）。

【输出描述】

> 结果（是负数要输出负号）。

【示例一】

> 输入：
>
> ```
> 2
> 1
> ```
>
> 输出：
>
> ```
> 1
> ```

【解法】

模拟小学「列竖式」计算「两数相减」的过程。



1. 用字符串读入数据；
2. 判断两个数的大小，让较大的数在前。注意字典序 vs 数的大小：
   a. 位数相等：按字典序比较；
   b. 位数不等：按照字符串的长度比较。
3. 将字符串的每一位拆分，逆序放在数组中；
4. 模拟列竖式计算的过程：
   a. 对应位求差；
   b. 处理借位；
5. 处理前导零。

【参考代码】

代码块

```
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int a[N], b[N], c[N];
int la, lb, lc;

// 比大小
bool cmp(string& x, string& y)
{
    // 先比较长度
    if(x.size() != y.size()) return x.size() < y.size();

```

```cpp
    // 再按照 字典序 的方式比较
    return x < y;
}

// 高精度减法的模板 - c = a - b
void sub(int c[], int a[], int b[])
{
    for(int i = 0; i < lc; i++)
    {
        c[i] += a[i] - b[i]; // 对应位相减，然后处理借位
        if(c[i] < 0)
        {
            c[i + 1] -= 1; // 借位
            c[i] += 10;
        }
    }

    // 处理前导零
    while(lc > 1 && c[lc - 1] == 0) lc--;
}

int main()
{
    string x, y; cin >> x >> y;

    // 比大小
    if(cmp(x, y))
    {
        swap(x, y);
        cout << '-';
    }

    // 1. 拆分每一位，然后逆序放在数组中
    la = x.size(); lb = y.size(); lc = max(la, lb);
    for(int i = 0; i < la; i++) a[la - i - 1] = x[i] - '0';
    for(int i = 0; i < lb; i++) b[lb - i - 1] = y[i] - '0';

    // 2. 模拟减法的过程
    sub(c, a, b); // c = a - b

    // 输出结果
    for(int i = lc - 1; i >= 0; i--) cout << c[i];

    return 0;
}
```

2.3 高精度乘法

> **题目来源：** 洛谷
>
> **题目链接：** P1303 A*B Problem
>
> **难度系数：** ★

【题目描述】

> 给出两个非负整数，求它们的乘积。

【输入描述】

> 输入共两行，每行一个非负整数。

【输出描述】

> 输出一个非负整数表示乘积。

【示例一】

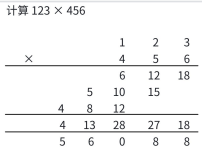
> 输入：
>
> 1
>
> 2
>
> 输出：
>
> 2

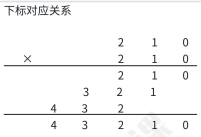
【解法】

无进位相乘再相加：

- 还是「列竖式」，但是每一位相乘的时候不考虑进位，直接把乘的结果放在对应位上；
- 等到所有对应位置「乘完」并且「累加完」之后，「统一处理进位」。

如下图所示：





1. 用字符串读入数据；
2. 将字符串的每一位拆分，逆序放在数组中；
3. 模拟无进位相乘再相加的过程：
    a. 对应位求乘积；
    b. 乘完之后处理进位；
    c. 处理余数；
4. 处理前导零。

【参考代码】

代码块

```cpp
#include <iostream>

using namespace std;

const int N = 1e6 + 10;

int a[N], b[N], c[N];
int la, lb, lc;

// 高精度乘法的模版 - c = a * b
void mul(int c[], int a[], int b[])
{
    // 无进位相乘,然后相加
    for(int i = 0; i < la; i++)
    {
        for(int j = 0; j < lb; j++)
        {
            c[i + j] += a[i] * b[j];
```

```
        }
    }

    // 处理进位
    for(int i = 0; i < lc; i++)
    {
        c[i + 1] += c[i] / 10;
        c[i] %= 10;
    }

    // 处理前导零
    while(lc > 1 && c[lc - 1] == 0) lc--;
}

int main()
{
    string x, y; cin >> x >> y;

    // 1. 拆分每一位，逆序放在数组中
    la = x.size(); lb = y.size(); lc = la + lb;
    for(int i = 0; i < la; i++) a[la - 1 - i] = x[i] - '0';
    for(int i = 0; i < lb; i++) b[lb - 1 - i] = y[i] - '0';

    // 2. 模拟乘法的过程
    mul(c, a, b); // c = a * b

    // 输出结果
    for(int i = lc - 1; i >= 0; i--) cout << c[i];

    return 0;
}
```

2.4 高精度除法

> **题目来源：** 洛谷
>
> **题目链接：** P1480 A/B Problem
>
> **难度系数：** ★

【题目描述】

> 输入两个整数 a, b ，输出它们的商。

【输入描述】

> 两行，第一行是被除数，第二行是除数。

【输出描述】

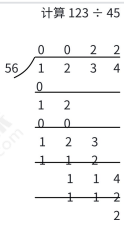
> 一行，商的整数部分。

【示例一】

> 输入：
>
> 10
>
> 2
>
> 输出：
>
> 5

【解法】

模拟小学「列竖式」计算「两数相除」的过程（注意，我们这里是「高精度 ÷ 低精度」）。



定义一个指针 i 从「高位」遍历被除数，一个变量 t 标记当前「被除的数」，记除数是 b ；

- 更新一个当前被除的数 $t = t \times 10 + a[i]$ ；
- t/b 表示这一位的商， $t\%b$ 表示这一位的余数；
- 用 t 记录这一次的余数，遍历到下一位的时候重复上面的过程

被除数遍历完毕之后， t 里面存的就是余数，但是商可能存在前导 0 ，注意清空。

【参考代码】

代码块

```
#include <iostream>

using namespace std;
```

```cpp

const int N = 1e6 + 10;

typedef long long LL;

int a[N], b, c[N];
int la, lc;

// 高精度除法的模板 - c = a / b （高精度 / 低精度）
void sub(int c[], int a[], int b)
{
    LL t = 0; // 标记每次除完之后的余数
    for(int i = la - 1; i >= 0; i--)
    {
        // 计算当前的被除数
        t = t * 10 + a[i];
        c[i] = t / b;
        t %= b;
    }

    // 处理前导 0
    while(lc > 1 && c[lc - 1] == 0) lc--;
}

int main()
{
    string x; cin >> x >> b;
    la = x.size();

    for(int i = 0; i < la; i++) a[la - 1 - i] = x[i] - '0';

    // 模拟除法的过程
    lc = la;
    sub(c, a, b); // c = a / b

    for(int i = lc - 1; i >= 0; i--) cout << c[i];


    return 0;
}
```

3. 枚举